

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Internet Programming
COURSE CODE: CSA43

NAME: Varsha S
REG NO: 192462003

 **Browser & Web Technologies**
HTTP • DOM • HTML • Browser Communication Analysis

Home Questions Summary

 **Scenario**

A web application is being analyzed to identify issues related to HTTP response headers, DOM structure, login methods, HTML syntax, and browser-server communication. The following analysis identifies the problems, explains their impact, and provides the correct solutions based on web development standards.

 **Question 1 – HTTP Response Analysis**

✓ Analysis Completed

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

- Identify any incorrect or invalid fields.**

The invalid field is **Content-Length: -150**. The Content-Length header must always contain a non-negative integer representing the size of the response body in bytes. A negative value is not permitted according to HTTP standards.
- Analyze the issue with Content-Length and explain its impact.**

The **Content-Length** header informs the browser about the exact size of the response body. A negative value causes the response to become invalid and may result in protocol errors, interrupted data transfer, incorrect page loading, or the browser rejecting the response.
- Determine whether the status code matches the response correctness.**

The status code **200 OK** correctly indicates that the server successfully processed the request. However, the response header is still considered incorrect because it contains an invalid **Content-Length** value. Therefore, the status code is correct, but the overall response is not fully valid.

4. Suggest corrections for the identified errors.

Replace the invalid Content-Length value with the correct size of the response body.

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2025 10:00:00 GMT
Content-Type: text/html
Content-Length: 158
Connection: keep-alive
```

This corrected header follows HTTP standards and ensures proper communication between the browser and the server.

Question 2 – DOM Structure Analysis

✓ DOM Analysis Completed

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

1. Identify missing or improperly closed elements.

The HTML contains improperly closed elements.

- The `<p>` tag is missing its closing tag `</p>`.
- The paragraph remains open when the `</div>` tag is encountered.
- This creates an invalid DOM structure.

2. Analyze how the browser will interpret this structure.

Modern web browsers automatically correct many HTML errors. During parsing, the browser inserts the missing closing `</p>` tag before closing the `<div>`. Although the page appears correctly, the browser modifies the original DOM internally.

3. Determine the impact on rendering and layout.

Because browsers perform automatic error correction, the webpage usually renders without major visual problems. However, invalid HTML may lead to inconsistent layouts, unexpected CSS behavior, JavaScript DOM manipulation issues, and reduced maintainability of the code.

4. Suggest corrections to make the DOM valid.

The corrected HTML structure is shown below.

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>
```

Question 3 – GET vs POST in Login System

✓ Login Method Analysis

Feature	GET	POST
Data Transmission	Data is appended to the URL. Data is sent inside the request body.	
Visibility	Visible in browser address bar	Not visible in URL
Security	Low	Moderate / Better
Suitable for Login	No	Yes

1. Interpret the differences in data transmission between GET and POST.

GET sends data by attaching it to the URL as query parameters, making it visible in the browser's address bar. POST sends data inside the HTTP request body, keeping it hidden from the URL and making it more suitable for transmitting sensitive information.

2. Analyze which method is more suitable for login and why.

POST is the preferred method for login systems because usernames and passwords are transmitted in the request body rather than appearing in the URL. This improves privacy and reduces the exposure of sensitive information.

3. Identify security risks in using GET for authentication.

- Passwords become visible in the URL.
- Credentials may be stored in browser history.
- Sensitive information can appear in server logs.
- Users may accidentally share login URLs.
- Search engines may index URLs containing sensitive information.

4. Suggest best practices for secure login implementation.

- Use the **POST** method for login forms.
- Always use **HTTPS** to encrypt communication.
- Hash and securely store passwords.
- Validate all user inputs.
- Implement secure session management.
- Enable multi-factor authentication (MFA) whenever possible.

Question 4 – HTML Code Inspection

✓ HTML Inspection Completed

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>

<body>

<h1>Welcome

<p>This is a paragraph



</body>
</html>
```

1. Identify missing closing tags.

- The **<h1>** tag is missing **</h1>**.
 - The **<p>** tag is missing **</p>**.
- The **** tag does not require a closing tag because it is a **void element** in HTML5.

2. Analyze issues with the **** tag.

The image tag is syntactically valid, but it is missing the **alt** attribute. The **alt** attribute provides alternative text if the image cannot be displayed and improves accessibility for screen readers.

3. Determine how browsers will handle these errors.

Modern browsers automatically insert the missing closing tags and continue rendering the webpage. Although the page usually appears correctly, invalid HTML may produce inconsistent layouts, reduce accessibility, and create unexpected behavior when JavaScript interacts with the DOM.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>

<head>
<title>Sample</title>
</head>

<body>

<h1>Welcome</h1>

<p>This is a paragraph.</p>



</body>
</html>
```

Question 5 – Browser–Server Communication Flow

✓ Communication Flow Analyzed

Browser–Server Communication Flow

Step	Description
1 User enters URL	The user types the website address into the browser.
2 DNS Lookup	DNS translates the domain name into an IP address.
3 HTTP Request	The browser sends an HTTP request to the web server.
4 Server Processing	The server processes the request and prepares the webpage.
5 HTTP Response	The server returns HTML, CSS, JavaScript and other resources.
6 Browser Rendering	The browser renders the webpage for the user.

1. Interpret each step in the communication flow.

The browser first receives the website URL from the user. DNS converts the domain name into an IP address. The browser then sends an HTTP request to the web server. The server processes the request, prepares the required resources, returns an HTTP response, and finally the browser renders the webpage.

2. Identify where delays can occur.

2. Identify where delays can occur.

- Slow DNS lookup
- Network latency
- Heavy server processing
- Large HTML, CSS, JavaScript or image files
- Slow browser rendering

3. Analyze the role of DNS in the process.

The Domain Name System (DNS) converts human-readable domain names such as **www.example.com** into numerical IP addresses that computers use to locate and communicate with web servers. Without DNS, browsers would not know where to send HTTP requests.

4. Suggest optimizations to improve performance.

- Use DNS caching.
- Enable browser caching.
- Compress files using Gzip or Brotli.
- Optimize images and multimedia.
- Use a Content Delivery Network (CDN).
- Minify HTML, CSS and JavaScript files.
- Reduce unnecessary HTTP requests.

Summary

Topic	Key Finding
HTTP Response	Content-Length must always be a non-negative integer.
DOM Structure	Properly closing HTML tags ensures a valid DOM.
GET vs POST	POST is the recommended method for login authentication.
HTML Inspection	Mixing tags should be corrected and images should include alt attributes.
Browser Communication	DNS, server performance and optimized resources improve webpage speed.

Conclusion

This case study demonstrates the importance of following web development standards when designing modern web applications. Correct HTTP headers, valid HTML and DOM structures, secure authentication methods, proper browser-server communication and performance optimization together improve security, accessibility, reliability and user experience. Following these best practices results in robust and standards-compliant web applications.